

Progress Report

TrioTagaPayo — AMS_DBTC Project

Date: April 25, 2026

Reporter: James Neil M. Maano

Project: TrioTagaPayo AMS DBTC

Report Period: March 7, 2026 – April 25, 2026

Sprint / Period: Current Sprint

STAND-UP REPORT

What was achieved this period?

- Set up the Backend folder/project structure
- Pushed code to GitHub repository and created a dedicated Backend branch
- Implemented clean code refactoring across backend files
- Updated and refined pagination functionality via PaginationHelper
- Added QR Code Attendance System
 - QRSession and QRScan models added
 - IQRSessionRepository interface defined with token lookup, scan tracking, and active session queries
 - QRSessionRepository implementation with full DB query support
 - IQRService interface designed with GenerateAsync, ScanAsync, DeactivateAsync, GetActiveSessionsAsync
 - AppDbContext updated: QRSessions and QRScans DbSet, relationships, and unique scan index
- Integrated EmailJS Email Notification System
 - EmailJSHelper implemented — sends attendance notification emails via EmailJS REST API
 - EmailJSOptions configuration class added and wired to appsettings
 - HttpClient registered with AddHttpClient() to avoid double-registration
 - Email failure is isolated — will never crash the attendance save flow
- Enhanced Attendance Repository
 - IAttendanceFilterRepository added — supports date-range filtering by course
 - IAttendanceBulkRepository added — supports bulk ID-based lookups
 - AttendanceRepository implements all three interfaces
- Added Global Exception Middleware
 - ExceptionMiddleware catches all unhandled exceptions and returns structured JSON error responses
 - Maps exception types to proper HTTP status codes (404, 400, 401, 500)
- Program.cs fully configured
 - All repositories and services registered with proper scoped lifetimes

- JWT Bearer authentication configured with zero clock skew
- Swagger/OpenAPI with Bearer security definition enabled
- CORS policy configured for AllowAll
- Admin seeding on startup via `IAuthService.SeedAdminAsync()`
- Implemented API Rate Limiting (NEW)
 - `RateLimitingHelper` added to Helpers folder using .NET built-in `AddRateLimiter`
 - Policy 1: 'login' — max 5 attempts per minute per IP (brute force protection)
 - Policy 2: 'global' — max 100 requests per minute for general API endpoints
 - Policy 3: 'qrscan' — max 10 scans per 30 seconds per IP
 - Custom JSON rejection response (HTTP 429) — not HTML
 - `[EnableRateLimiting]` applied to Login and QR Scan endpoints
- Implemented Refresh Token with Rotation (NEW)
 - `RefreshToken` and `RefreshTokenExpiry` fields added to User model
 - EF Core migration applied: `AddRefreshTokenToUser`
 - `RefreshTokenHelper` added — generates 64-byte cryptographically secure random token
 - `LoginResponse` updated to include `RefreshToken` and `RefreshTokenExpiry`
 - `RefreshTokenRequest` model added for POST `/api/Auth/refresh`
 - `IAuthService` updated with `RefreshAsync` method
 - `AuthService.RefreshAsync` implemented — validates token, rotates on use, old token invalidated
 - POST `/api/Auth/refresh` endpoint added to `AuthController`
- Implemented SignalR Real-time Notifications (NEW)
 - `AttendanceHub` added — supports `JoinCourseGroup`, `LeaveCourseGroup`, `JoinAdminGroup`
 - `AttendanceNotificationDto` added for real-time payload structure
 - `AddSignalR()` registered in `Program.cs`; hub mapped at `/hubs/attendance`
 - JWT from query string supported for WebSocket connections (SignalR requirement)
 - `AttendanceService.NotifyHubAsync` fires on every manual attendance save
 - `QRService.NotifyHubAsync` fires on every successful QR scan
 - Broadcasts to `course_{courseId}` group (teacher) and admin group (global feed)
 - `Source` field distinguishes 'manual' vs 'qr_scan' notifications
- Implemented JWT Cookies / Session (NEW)
 - Login endpoint sets `HttpOnly + Secure + SameSite=Strict` cookies for `accessToken` and `refreshToken`
 - `refreshToken` cookie path restricted to `/api/Auth/refresh` only
 - Refresh endpoint auto-detects token from cookie (browser) or JSON body (mobile)
 - POST `/api/Auth/logout` endpoint added — deletes both cookies
 - GET `/api/Auth/me` endpoint added — returns `userId`, `username`, `role` from JWT claims
 - `ClaimsHelper` added — `User.GetUserId()`, `GetUsername()`, `GetRole()` extension methods
 - `JwtBearerEvents.OnMessageReceived` updated — priority: cookie > SignalR query > Bearer header

What is the focus today?

- Deploy backend API to Railway or Render for live public access
- Configure environment variables (JWT key, DB connection string, EmailJS keys) in cloud platform
- Run EF Core migrations against cloud PostgreSQL database
- Verify all endpoints working on live URL via Swagger
- Test SignalR connection from frontend to deployed backend
- Test refresh token flow and cookie behavior on deployed environment

What blockers exist?

- API still running on localhost — not yet deployed to cloud platform
- Cloud PostgreSQL not yet provisioned — needs Neon, Supabase, or Railway PostgreSQL
- Environment variables not yet configured in deployment platform
- SignalR WebSocket behavior on cloud hosting needs verification

RETROSPECTIVE

What went well? (Wins/Successes)

- QR-based attendance feature is well-scoped with clean separation of models, repo, and service layers
- EmailJS integration was added without coupling it tightly to the attendance save — failure-safe design
- Repository pattern is now more flexible with the addition of filter and bulk interfaces
- Global exception middleware eliminates boilerplate try/catch across all controllers
- Backend folder structure and Git branching remain organized
- Pagination helper is now reusable across all services
- Rate limiting successfully protects login and QR scan endpoints from brute force and spam
- Refresh token rotation ensures old tokens are immediately invalidated after use — strong security posture
- SignalR integration cleanly separated into NotifyHubAsync helpers — reused in both AttendanceService and QRService
- JWT cookie implementation supports both browser (HttpOnly cookie) and mobile (Bearer token) clients simultaneously
- ClaimsHelper extension methods make it clean and consistent to read user identity across all controllers

What didn't go well? (Problems/Challenges)

- Deployment not yet complete — API is still on localhost
- IUserRepository was missing FindAsync and AnyAsync — caused CS1061 compile error, resolved by verifying GenericRepository base methods
- CS0108 warnings appeared when re-declaring methods already in GenericRepository — resolved by removing duplicate declarations
- SignalR WebSocket behavior on cloud hosting not yet verified

What should we start doing? (Improvements)

- Deploy to Railway or Render immediately — this is the last remaining hard requirement
- Write integration tests for refresh token rotation and cookie behavior
- Test SignalR real-time notifications end-to-end with frontend
- Document new endpoints (refresh, logout, me) in Swagger XML comments

What should we stop doing? (Waste/Inefficiencies)

- Avoid leaving DI registrations incomplete after implementing new services
- Avoid committing untested code changes — test after each feature addition
- Avoid re-declaring methods in child repositories/interfaces that are already in the generic base

What should we continue doing? (Best Practices)

- Maintain organized folder structure and feature-branch Git workflow
- Continue using repository pattern with interface-based DI for testability
- Keep email and notification failures isolated from core business logic
- Continue consistent progress updates and stand-up check-ins
- Keep helper classes (ClaimsHelper, RefreshTokenHelper, RateLimitingHelper) in the Helpers folder for consistency

Submitted by:

James Neil M. Maano

TrioTagaPayo Team

March 7, 2026 – April 25, 2026